ARCHITECTURE BRIEF / 01

# MedLiveBD Project architecture

A codebase-grounded briefing for a prospective business and technical partner | 16 October 2026

A healthcare coordination platform for Bangladesh: patients discover care and book a serial; hospital staff operate the queue and related workflows. The central promise is that reception calls the next patient and waiting patients see the change within two seconds. This is a product requirement, not a measured guarantee for every deployment. [S1, S4]

### Current implementation and requested change

**Current:** TypeScript, Express, Socket.IO and PostgreSQL.
**Requested target:** Django REST Framework (DRF). The backend migration has not been implemented.
**Local tools:** the desktop launcher UI and its shortcuts have been removed.

## What kind of architecture is it?

A **pnpm monorepo** containing three web apps, shared packages and a **modular monolith backend**. A modular monolith means one API deployment holds several well-separated business modules. It is not a collection of independent microservices. [S2, S3]

| Product surface | What the user does |
|---|---|
| Patient app | Search facilities and doctors; book; track a live serial; access permitted records and care workflows. |
| Hospital console | Reception, doctor, ward, ER, lab/pharmacy, setup, imports and administration within a hospital workspace. |
| Company website | Explain the product, link to the demonstration and collect enquiries through contact links. |

## Why it was designed this way

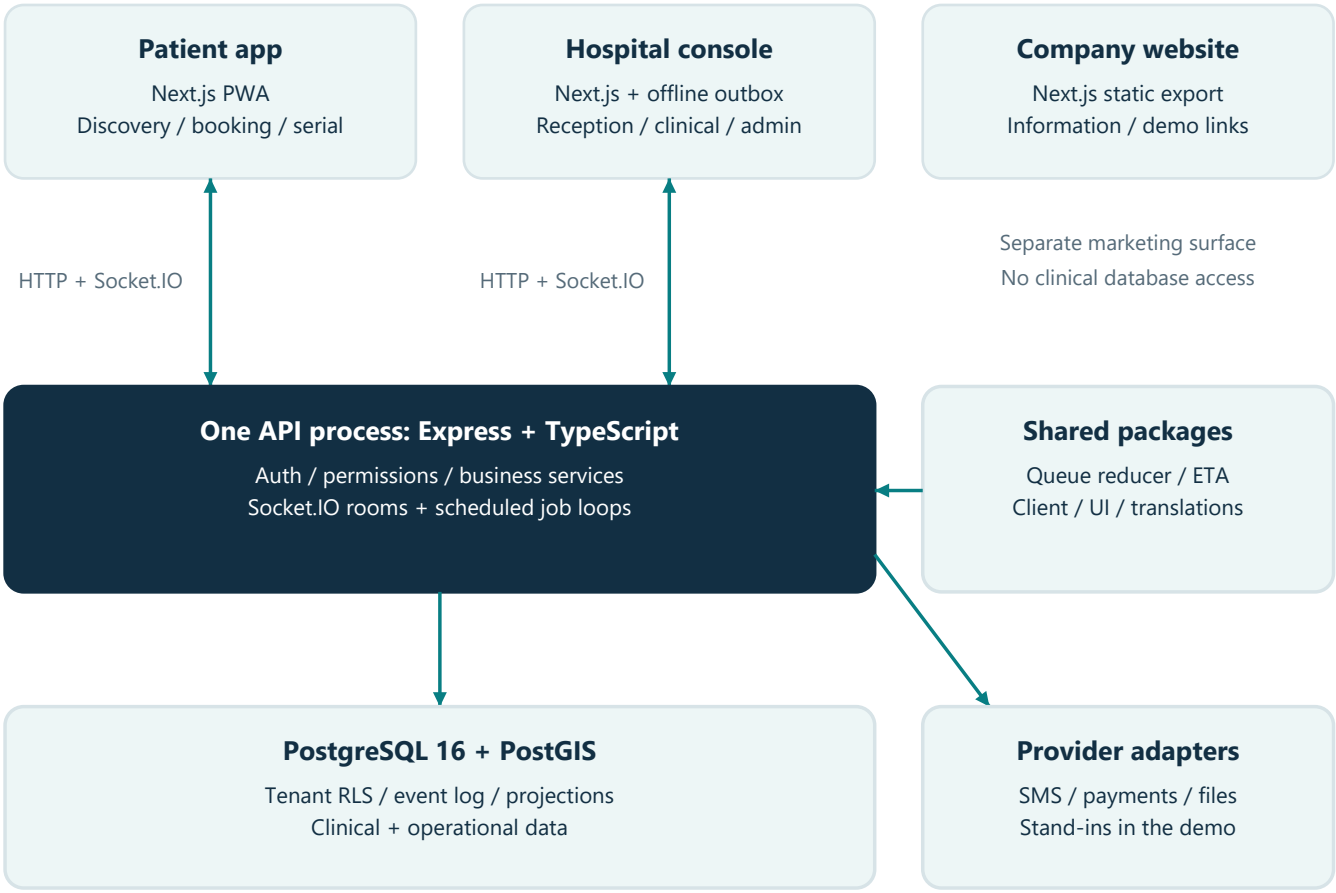
One shared platform connects participating hospitals while keeping their private data separate. Shared queue rules support optimistic staff screens; an append-only event history supports replay; realtime updates reduce uncertainty; freshness stamps show the age of operational figures. These are the design goals visible in the requirements and code. [S1-S5]

Scope: current code was inspected in the original checkout and the selected newer hospital-app worktree. No public deployment, cloud account or full test suite was revalidated for this PDF. Source and branch qualifications are recorded on pages 8 and 10.

ARCHITECTURE BRIEF / 02

# The system at a glance

Solid arrows show application communication or a code dependency; provider behavior depends on configuration. [S2-S6]




## How the pieces communicate

**HTTP:** frontends call versioned endpoints under `/api/v1`. **Realtime:** Socket.IO pushes queue and operational events to authorized rooms, with WebSocket and polling transports. **Database:** the API owns business writes through repositories. Browser apps do not connect directly to PostgreSQL.

**Shared packages are code, not servers.** The API and staff console import the same TypeScript queue reducer and ETA rules. The client package handles typed requests, socket subscriptions and an IndexedDB-backed offline queue. UI components and translations are shared separately. [S4, S6]

### Important implementation distinction

Background work currently runs **inside the API process**. The `backend/workers` package is a stub; the described pg-boss worker deployment is not implemented. Redis and a separate job service are not prerequisites of the current runtime. [S3]

ARCHITECTURE BRIEF / 03

# Code structure and API layers

The folder structure separates browser code, server code, reusable logic and database ownership. [S2-S4]

| Location | Responsibility |
|---|---|
| frontend/patient | Next.js patient PWA; patient-facing discovery, booking, serial and records. |
| frontend/console | Next.js hospital workspace; staff screens and supported offline actions. |
| frontend/site | Marketing website, including a static export / Cloudflare Pages release path. |
| backend/api | Express HTTP API, Socket.IO, authentication, services, repositories and job loops. |
| shared/domain | Pure queue reducer, event types, ETA/rules, schemas and business types. |
| shared/client, ui, i18n | API/realtime/offline clients, common components and bn/en translations. |
| database | Forward SQL migrations, synthetic seeds, schema checks and API-role provisioning. |
| deploy + .github/workflows | Self-hosted containers, TLS/backup scripts and CI definitions. |

## A request travels through five layers

**Route** matches the URL and applies guards/validation. **Controller** translates the HTTP request and response. **Service** applies the business operation and coordinates transactions. **Repository** reads and writes rows. **Database** enforces constraints and row policies. Import/layering rules help keep these responsibilities separate. [S3]

## Shared HTTP contract

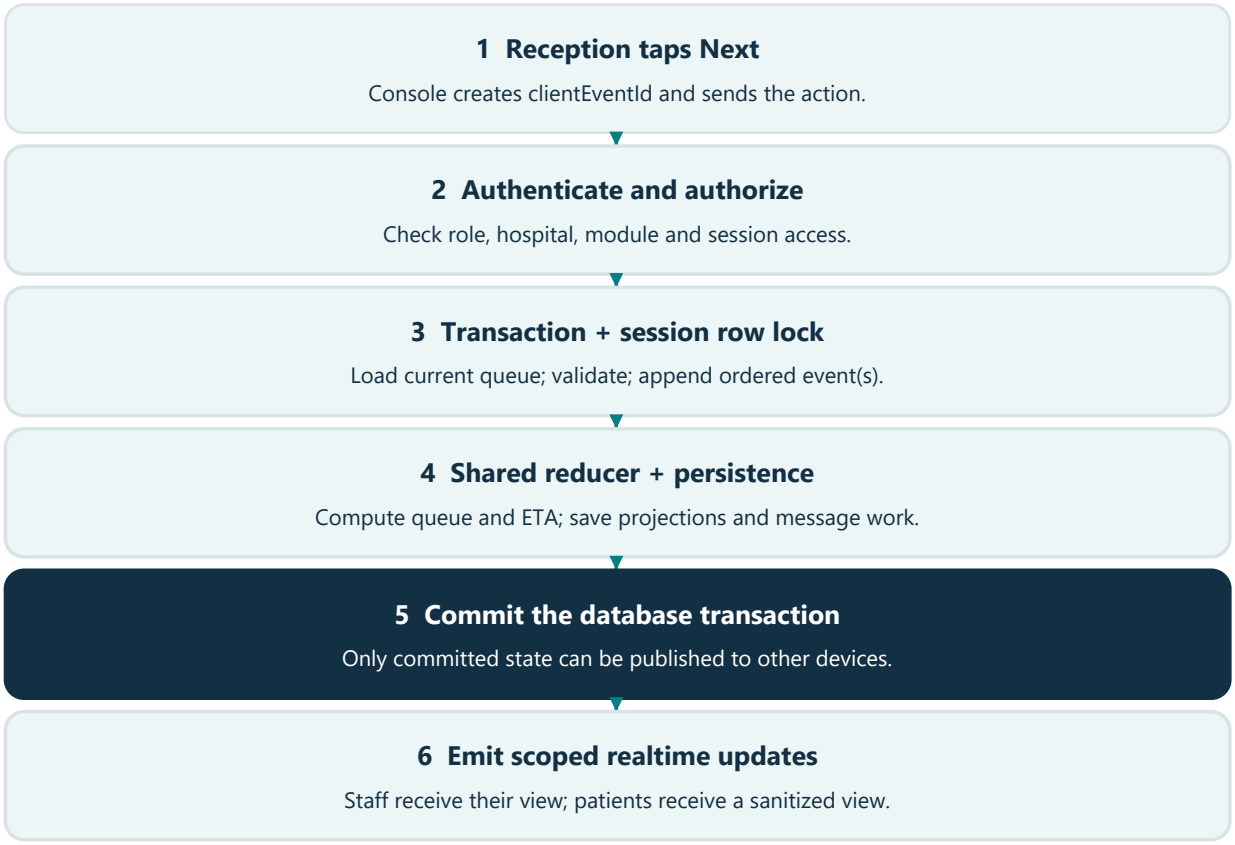
Success: `{ ok: true, data: ... }`
Failure: `{ ok: false, error: { code, message, ... } }`
The API includes discovery, booking, queue, auth, registration, clinical/consent, beds, emergency/referrals, lab, payments, hospital setup/import, platform administration, webhooks and offline sync. [S3]

Middleware performs request logging, security headers, CORS, bounded parsing, principal/guest attachment, request scope, module gating and idempotency checks before route handlers. Readiness and liveness probes live at /readyz and /healthz.

ARCHITECTURE BRIEF / 04

# The live queue: the core engine

The queue is event-based; this does not mean every module uses full event sourcing. [S4, S5]




## Why the lock, log and reducer matter

The **session row lock** serializes competing counter actions. The **append-only event log** preserves what happened and supports reconstruction. The **projection** stores derived state for efficient reads. The **clientEventId** identifies a retry so supported actions can be replayed without being performed twice.

## Offline and reconnect behavior

The console can show an optimistic result and store supported pending actions in IndexedDB. On reconnect it submits them for server validation and reconciliation. Sequence information and applied action IDs help the client identify what the server has accepted. Offline does not make every workflow available without the API. [S5, S6]

### A reliability boundary to preserve in DRF

Committing before broadcasting prevents clients seeing rolled-back state. A process crash **after commit but before delivery** can still lose a live notification. Replay/reconnect behavior and, where required, a durable outbox need explicit tests; a framework change alone does not close that window.

ARCHITECTURE BRIEF / 05

# Data, tenancy and access control

The platform shares infrastructure; each caller must still see only the data they are allowed to access. [S3, S5, S7]

| Data family | Examples and role |
|---|---|
| Organization / setup | Hospitals, departments, staff roles, modules, branding, schedules and capabilities. |
| Queue / booking | Sessions, bookings, queue events and derived state used by live serial screens. |
| Patient / clinical | Accounts/profiles, visits, prescriptions, tests, reports, documents and consent. |
| Operations / integration | Beds, emergency/referrals, payment state, notifications, imports and audit evidence. |

## Two layers protect tenant boundaries

**Application permissions** check who may perform an action and for which hospital or patient. **PostgreSQL row-level security (RLS)** also restricts visible rows. Request context is carried through AsyncLocalStorage and stated on checked-out database connections using app.scope, app.hospital_id, app.person_id and app.booking_id.

Scopes include hospital, patient, guest, national, open and system. The API uses a limited role; migrations and provisioning use a separate owner connection. A hospital must not be selected merely from a caller-supplied field. Some carefully controlled queue operations temporarily use server scope after application authorization. [S5, S7]

## Authentication and sensitive data

Implemented mechanisms include JWT access/refresh flows, scrypt staff passwords, staff administrator TOTP and signed guest tracking links. Patient phone verification depends on the configured SMS path. Socket authentication and room admission are separate checks; patient queue views are sanitized instead of exposing the full staff queue. [S3, S5]

Clinical files use storage adapters and signed, expiring access links. The local production-stack design uses persistent files. Supabase storage configuration is refused in production on the selected audit branch because an implemented adapter is missing. [S8]

### Design intent is not a security certification

RLS, role tests and audit code provide useful controls. Production readiness still needs policy decisions, failure tests, deployment hardening and proof that the deployed API actually uses the limited database role. Only synthetic demonstration data should be used in the demo. [S1, S7, S8]

ARCHITECTURE BRIEF / 06

# How the product should work

A practical walkthrough from hospital setup to the patient seeing their serial change. [S1, S3-S6]

**01 Onboard and configure a hospital**
A platform administrator reviews the organization. Hospital administrators configure departments, doctors, schedules, staff, fees, capabilities, modules and branding. Publication determines what the patient network can discover.

**02 Discover and book care**
A patient searches the network, chooses a doctor/session and books a place. Guest, telephone and counter paths feed the same session workflows. A guest tracking link grants bounded access; public operational figures display their freshness.

**03 Operate the chamber**
Reception checks in patients, calls Next and handles delays, late arrivals, cancellations and no-shows. The API validates and records the action, then publishes the permitted queue view to staff and waiting patients.

**04 Complete care and share permitted records**
Staff complete visits and the relevant clinical workflows. Patients access their own records. Cross-hospital clinical access follows the consent/referral model rather than treating network participation as unrestricted record sharing.

**05 Keep operations current**
Ward, ER and lab/pharmacy staff update their workflows. API loops materialize scheduled sessions, expire queue offers/payment holds and send or retry notifications. Administrators review operational data.

## External integrations: code versus activation

| Integration | What the repository establishes |
|---|---|
| SMS / patient OTP | Log/demo behavior and an HTTP provider adapter. Real delivery and sender approval need external activation. |
| Payments | Mock/off behavior and provider adapters. Live merchant approval and provider verification are separate. |
| Maps / import AI | Static travel-time mode; optional column-mapping suggestions that require human confirmation. |
| Hospital systems | No verified live HMS or national-record integration established by this review. |

The system coordinates care workflows. It is not a complete hospital ERP: payroll, HR, general accounting, insurance administration and PACS are outside the declared scope. [S1]

ARCHITECTURE BRIEF / 07

# Deployment, local runtime and CI/CD

Configuration and dated review evidence are distinguished from a fresh inspection of cloud accounts. [S9-S12]

| Component | Recorded deployment path |
|---|---|
| Company website | Cloudflare Pages; medlivebd.com / www.medlivebd.com and medlivebd.pages.dev. Manual preview and confirmed production-upload script. |
| Patient / console demos | Vercel: healthcare-patient-zeta.vercel.app and healthcare-console.vercel.app, recorded in the project review. |
| API demo | Render: healthcare-api-m9ke.onrender.com; blueprint targets main and a demonstration configuration. |
| Demo database | Supabase, documented as Singapore. Cloud settings and ownership are not verified by this PDF. |
| Real-data hosting design | Self-hosted Docker stack intended for a Bangladesh host; a live hospital deployment is not established here. |

## Self-hosted stack in the repository

Caddy provides TLS/routing to patient, console and API containers. PostgreSQL/PostGIS starts first; a one-shot migration/role service prepares the schema; API and web apps follow health checks. Persistent files and a scheduled backup/restore-check service are included. The company website has a separate release path. [S10]

## CI exists; automatic deployment gates need confirmation

GitHub Actions runs on pushes and pull requests targeting main/mvp. Jobs cover type/lint/format checks, database and application tests, builds, the two-device queue canary, browser flows and production-mode tests with a limited database role. The workflow is a test gate, not itself the application deployment engine. Provider-managed deploy settings and branch protection must be checked to prove red builds cannot release. [S9, S11]

## Local development on this machine

`.\run-latest.ps1` starts the selected newer API/patient/console plus the original checkout's newer site. Ports: patient 3000, console 3100, site 3200, API 4000. Its `-Stop` option stops app processes and preserves the database. Local PostgreSQL uses Ubuntu WSL on this machine; Docker is another supported setup path. The desktop GUI is removed. [S12, S13]

Deployment addresses above are from the dated repository review (14 October); no public endpoint, dashboard, current bill, region or live configuration was rechecked for this document.

ARCHITECTURE BRIEF / 08

# Branch state and architecture gaps

The newest commit date alone does not identify the most complete runnable product. [S8, S12-S14]

| Source | Meaning for this review |
|---|---|
| mvp / cd2a676 | Original checkout baseline: newer company-site work; not all redesigned hospital-app changes. |
| audit/pilot-readiness-2026-10-12 / b7bee62 | Selected newer hospital-app candidate in .local/latest-app. Includes console redesign and several audited fixes. |
| audit/f17 / d97dc34 | Newer hospital-code timestamp, but explicitly unfinished staff-refresh WIP; not selected for routine startup. |
| main + demo / 99e4c5e | Older released demo baseline. The combined local preview is not an integrated production release. |

## What needs attention before a real pilot

**Operations:** the API is a single failure domain for HTTP, realtime and job loops. Multiple replicas need shared realtime/coordination and deliberate job ownership. The newer audit records backup encryption, host hardening and off-host alerting as open infrastructure work. [S3, S8]

**Policy and activation:** the selected branch's audit records unresolved lab pricing, breadth of doctor record access, national aggregation suppression, and clinical retention/erasure policy. Real SMS is required for non-demo patient entry; provider stand-ins do not prove live readiness. [S8]

**Correctness and recovery:** preserve atomic queue writes, same-action retry handling, tenant scoping and sanitized responses. Test external-call failures, concurrent operations and restart/reconnect behavior. Some limits and delivery paths are process-local. [S3, S5, S14]

### Do not mix findings from different branches

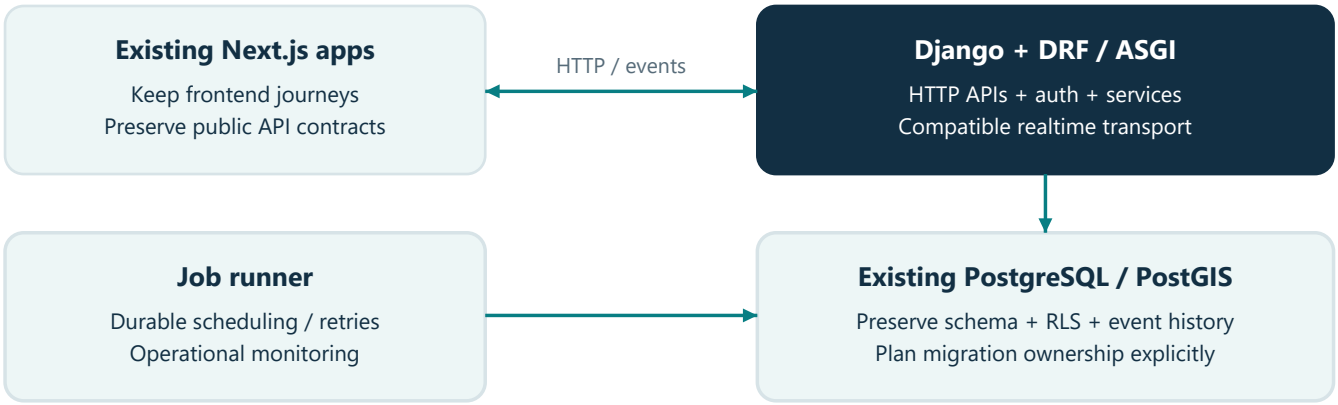
The separate 15 October **mvp** audit reproduced refund races, OTP concurrency and patient-session revocation gaps. Those probes were not rerun against b7bee62 for this PDF. Conversely, b7bee62 already contains fixes for patient queue response privacy, replay-key scope and integration environment passthrough. [S8, S12, S14]

**Evidence limits:** prior local startup smoke checks passed for all four services. The selected branch records its own tests and a synthetic restore drill. Its audit documents contain stale checklist/verdict fields; no new full release gate was run for this report. Treat production readiness as unconfirmed. [S8, S12]

ARCHITECTURE BRIEF / 09

# Requested direction: Django + DRF

Proposed migration architecture and work sequence. None of this is represented as already implemented.




## What should stay consistent

Keep PostgreSQL/PostGIS, event history, tenant boundaries, idempotency and existing user journeys. Preserve HTTP response/error shapes while the frontends still use them. DRF replaces the HTTP framework; it does not automatically replace Socket.IO, scheduled work, queue rules or database access policies.

| Step | Concrete work and completion gate |
|---|---|
| 1. Contract inventory | Map every endpoint, permission, schema, error, job and socket event. Decide one integration branch and migration ownership. |
| 2. DRF foundation | Create the Python environment, settings, ASGI server, envelopes, authentication, request scope and health/readiness checks. Test denial paths. |
| 3. Data and queue parity | Map the existing schema without destructive recreation. Preserve RLS and locks. Use common fixtures to prove Python/TypeScript reducer equivalence. |
| 4. Migrate workflows | Move discovery, booking, hospital operations, clinical/consent, imports and money paths module by module with contract/security tests. |
| 5. Realtime and jobs | Retain Socket.IO compatibility or update both clients deliberately; implement durable job scheduling/retries and test after-commit delivery. |
| 6. Cut over and remove Express | Update run scripts, CI, containers and deployment settings. Pass tenant, offline, queue canary and restore tests before switching all traffic. |

### The difficult part is preserving behavior

Python cannot directly import the current TypeScript reducer. A tested Python equivalent or an explicit shared-domain strategy is required. Existing clients speak the Socket.IO protocol; a plain WebSocket endpoint is not a drop-in replacement. These are migration design decisions, not missing DRF settings.

ARCHITECTURE BRIEF / 10

# Source map and reading order

References are relative to V:\STARTUP\HealthCare. No secrets or environment values are included.

For hospital application code, references S3-S8 use the selected **.local/latest-app** worktree unless stated otherwise. Company-site and local setup references use the original checkout. This distinction is necessary because there is no single branch containing all reviewed latest components.

| ID | Source and what it establishes |
|---|---|
| S1 | CLAUDE.md; docs/PRD.md; docs/PROJECT_REVIEW.md - product intent, demo boundaries and business context. |
| S2 | README.md; pnpm-workspace.yaml; package.json; frontend/*/package.json - workspace layout and frontend stack. |
| S3 | backend/api/package.json; src/app.ts; src/server.ts; src/routes/index.ts; backend/workers/src/index.ts - runtime, modules and actual jobs. |
| S4 | shared/domain/src/queue/; shared/domain/package.json - reducer, events, replay, rules and ETA. |
| S5 | backend/api/src/services/queue.service.ts; repositories/transaction.ts; realtime/server.ts, rooms.ts - atomic writes and broadcasts. |
| S6 | shared/client/package.json and src/; frontend/console/src/ - typed client, Socket.IO and offline queue. |
| S7 | backend/api/src/config/db.ts, dbScope.ts; database/migrations/; docs/DATABASE.md - pooled scope, schema and RLS. |
| S8 | docs/audit/pilot-readiness-2026-10-12/README.md, FINDINGS.md, CHECKLIST.md - newer-branch audit and its limitations. |
| S9 | .github/workflows/ci.yml - CI triggers, verification and browser jobs (original checkout). |
| S10 | deploy/docker-compose.yml; Dockerfile; docs/DEPLOY.md; deploy/backup.sh and restore.sh - self-hosting and recovery design. |
| S11 | render.yaml; frontend/site/scripts/deploy-pages.mjs; docs/PROJECT_REVIEW.md - recorded demo hosting and site release process. |
| S12 | docs/BRANCH_REVIEW.md - fetched branch comparison, selected commits and combined local preview, 16 October. |
| S13 | run.ps1; run-latest.ps1; scripts/dev-service.mjs; docs/STATUS.md - local startup and confirmed launcher removal. |
| S14 | docs/FLOW_ARCHITECTURE_AUDIT.md - separate mvp audit, 15 October; not evidence of unchanged defects on b7bee62. |

**Suggested reading order:** product intent (S1), system/runtime (S2-S3), one queue action (S4-S6), data isolation (S7), newer audit and branch state (S8/S12), then deployment and release controls (S9-S11).